Optimization, Security and Missing-Pieces Review

Project: Cashflow application — ADT Dental Clinic
Date: October 27, 2026
Scope: the code in the repository (performance and rendering, security, compliance and product basics). Read-only review: nothing in the application or the database was changed, except that a suggested SQL file was written (not run).

---

1. Summary — what to do first

The application is in good shape for its size: every server action checks the sign-in, passwords are hashed with bcrypt, input is validated, there is an audit trail, lists are paginated, the login now expires, and uploads are size- and type-limited. The gaps below are the usual ones for a system that grew feature by feature. Ranked by how much they matter:

#	What	Why it matters	Effort
1	**Upgrade Next.js (16.3.4 → 16.4.3)** and Drizzle (0.36 → 0.45.4)	A dependency scan reports **3 critical and several high** problems in the installed Next.js (remote code execution, a denial of service through Server Actions, cache poisoning). Fixed from 16.4.1.	S (half a day incl. testing)
2	**Let people change their own password**; replace the placeholder "Forgot password"	There is **no change-password feature**: the temporary password an admin types stays the password forever. The forgot-password page only pretends to send an e-mail.	M
3	**Add a Privacy Policy, Terms and a privacy notice** (Philippine Data Privacy Act)	The system holds patient names, birthdays, contact numbers, HMO account numbers, PWD / Senior IDs and visit history: health-related personal data.	M (needs the clinic's / a lawyer's input)
4	**Login throttling + security headers**	No rate limit or lock-out on sign-in; no CSP, HSTS, frame or content-type headers.	S–M
5	**Add the missing database indexes**	Day end, the transactions list and reports filter on columns with no index. Fine today, slow as data grows. SQL is ready (`2026-10-performance-indexes.sql`).	S
6	**Stop sending the whole database to the Overview page**	The dashboard loads *every* transaction and expense and ships them to the browser. It gets slower every month and every staff member's browser receives all branches' figures.	M
7	**Separate test data from live data**	The app on the developer's machine talks to the same Neon database (test visits were saved in it). Use a Neon branch for development.	S
8	**Backups, monitoring, error pages**	No `error.tsx` / `not-found.tsx`, no error monitoring, no documented restore.	S–M
9	**Move uploaded files out of the database**	Files are stored as base64 text in Postgres (+33 % size, loaded fully into memory per view).	M
10	**FAQ / Help page and in-app guide**	Staff questions (LOA pending, queue, day end, HMO) are answered in meetings only.	S–M

---

2. Security

2.1 Dependencies (do first)

`yarn audit` on the project's lock file (run today) reports 36 findings: **3 critical, 19 high, 13 moderate, 1 low**. The ones that matter for this application:

Package	Installed	Fixed in	Problem
`next`	16.3.4	**≥ 16.4.1** (latest 16.4.3)	Critical: remote code execution via the image-optimization API, `next/og` and `use cache`; High: denial of service in App Router **Server Actions** (the app uses many) and SSRF in image optimization; Moderate: cache poisoning and information disclosure
`drizzle-orm`	0.36.x	≥ 0.45.2 (latest 0.45.4)	High: SQL injection through improperly escaped *identifiers* (this app only uses fixed column names, so practical exposure is low, but upgrade)
`sharp` (via Next)	—	≥ 0.35.5	High (image library); comes along with the Next upgrade
`zod`, `brace-expansion`, `source-map-js`, `esbuild`	—	various	Mostly build-time / lint-time packages; they clear with a normal `yarn upgrade`

Several of the Next.js advisories concern features this app does not use (image optimization, `use cache`, `next/og`), so the real exposure is lower than "critical" suggests, but the Server Actions denial of service does apply, and an unpatched framework is the first thing a security review would flag.

How: on your machine run `yarn add next@16.4.3 eslint-config-next@16.4.3 drizzle-orm@latest`, then `yarn upgrade`, then `yarn build` and click through the main screens (login, Transactions, Queue, Day end, Patients). Read the Next 16.4 release notes first: the project's `AGENTS.md` correctly warns that this Next.js differs from older versions. Add **Dependabot or Renovate** so this is a monthly routine, and `yarn audit` in the build.

2.2 Accounts and sign-in

Finding	Detail	Suggestion
No change-password	Users cannot change the password an admin gave them; the "My profile" page has no password section.	Add Profile → Change password (current + new). Force a change at first sign-in (a `must_change_password` flag set when an admin creates the user or resets).
Forgot password is a placeholder	`forgot-password-form.tsx` has `// TODO` and a timer: it tells people "you'll receive instructions" but nothing is sent.	Either remove the link until it works, or build it: single-use token (hashed in the DB), 30-minute expiry, e-mail through Resend / Postmark / SES. Interim: change the page text to "Ask your admin to reset your password" and give admins a **Reset password** button on the Users page (generates a temporary password + forces change).
No rate limiting / lock-out	Unlimited password attempts.	Count failures per e-mail and per IP (a small `login_attempts` table or Upstash Redis); after 5 failures lock for 15 minutes; same error message for unknown e-mail and wrong password (already the case).
Weak password rule	Minimum 8 characters, no other check. The shared test superuser account used for browser testing must	Minimum 10–12, block the 10,000 most common passwords (e.g. `zxcvbn` or a small blocklist).

Finding	Detail	Suggestion
	be deleted (or given a strong password) before the system is used with live data on the internet.	
No sign-in history	Failed and successful sign-ins are not recorded; the audit trail covers data changes only.	Log sign-in, failed sign-in, sign-out, password change, role change, user created / disabled into `audit_log` (with IP and browser).
No two-step verification	A stolen password is enough.	Optional TOTP for admins first (the role that can delete and see IDs).
Users cannot be disabled or have their password reset by an admin	The Users page can create, change role / branch and delete only (no `active` flag, no reset action). Deleting is the only way to stop access (a removed user is signed out at the next refresh).	An `active` flag so staff who leave are blocked without deleting history.
`middleware.ts`	Next 16 renamed the middleware convention to *proxy*; the old name still works but is deprecated.	Rename when upgrading (see the Next 16 upgrade guide in `node_modules/next/dist/docs`).

2.3 Access control and data exposure

Good: every server action and every API route calls `auth()`; admin-only pages (Users, Branches, Commissions, Edit transaction) check the role on the server; Day end locks staff to their own branch; PWD / Senior ID numbers are masked for non-admins.

Finding	Suggestion
Overview sends everything to the browser (`src/app/cashflow/page.tsx` loads all transactions and expenses, all branches, and passes them to a client component). Any signed-in staff member's browser receives the full revenue history of every branch, even if the screen filters it.	Compute the dashboard figures in SQL (grouped by month / branch / dentist) and send only the totals; scope to the user's branch unless admin. Also fixes performance (section 3).
Attachments are readable by any signed-in user who has the link (`/api/cashflow/attachments/[id]` checks sign-in only). IDs are random UUIDs, so guessing is impractical, but a staff member of another branch can open a receipt if they have the id.	Check the file's visit / branch against the user's branch (or admin). Add `X-Content-Type-Options: nosniff` and `Content-Disposition: attachment` for PDFs. Verify the file's real type from its first bytes, not the browser-supplied type.
New patient details are personal data (birthday, HMO account number). They are shown to every signed-in role.	Mask the HMO account number (last 4) and show the birthday only to admins and the front desk; or keep as is if every role needs them for billing. Decide and document.
Patient ID numbers	Already masked for non-admins (good). Consider hiding the full number from admins too after entry, and log each reveal.
Deleting is permanent	Hard delete of visits (admin).
Branch scoping on the Transactions and Patients lists	Staff can look at all branches via the filter.

2.4 Browser protections (HTTP headers)

`next.config.ts` is empty, so none of the standard headers are sent. Add (via `headers()` in `next.config.ts`):

- `Content-Security-Policy` (start in *report-only* mode: `default-src 'self'; img-src 'self' data: blob:; script-src 'self' 'unsafe-inline'` and tighten with nonces later)
- `Strict-Transport-Security: max-age=63072000; includeSubDomains` (only once the site is HTTPS-only)
- `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (or `frame-ancestors 'none'`), `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: camera=(), microphone=(), geolocation=()`
- `poweredByHeader: false`
- `X-Robots-Tag: noindex` on every page plus a `robots.txt` that disallows everything: a private clinic system should not be indexed (the public login page at least).

The session cookie is set by Auth.js (HttpOnly, SameSite=Lax, Secure on HTTPS): keep `AUTH_URL` / `AUTH_TRUST_HOST` set correctly in production so the `Secure` flag is applied.

2.5 Secrets, environments, database

1. **Rotate the Neon password and `AUTH_SECRET`** (the old `.env.example` contained real values; this is still open from earlier).
2. **Separate development from live.** Create a Neon *branch* (or a second project) for development and testing, and point `.env.local` at it. Test visits and the test account then never touch clinic data, and a mistaken `db:push` cannot hurt production.
3. **Least-privilege database role** for the running app (no `DROP` / schema changes); keep the owner role for migrations only.
4. **Index drift:** the indexes created by the SQL files are not declared in `src/lib/db/schema.ts`. A future `drizzle-kit push` may propose *dropping* them. Declare every index in the schema (or stop using `push` and move to versioned migrations: `drizzle-kit generate` + `migrate`, which also gives you a history).
5. **Backups:** confirm Neon's point-in-time-restore window for your plan (the free plan is short) and set a scheduled export (`pg_dump` to a private bucket) weekly; **do a test restore once**. Write the steps in a runbook.
6. **Environment checks:** fail the start-up if `AUTH_SECRET` is missing or short (`DATABASE_URL` is already checked).
7. **Logging:** only 6 `console` calls exist. Make sure none prints patient data; send errors to a monitoring service (section 4).

2.6 What was checked and is fine

No `dangerouslySetInnerHTML`, `eval` or dynamic code; database access uses Drizzle's parameterized queries; uploads are limited to 4 MB and four types; file names are sanitised in downloads; server actions check the role; sessions expire after 30 idle minutes (12 h cap); the login form and server action do not reveal whether an e-mail exists.

---

3. Performance and rendering

3.1 Biggest items

#	Finding	Evidence	Fix
1	**Overview loads the entire `transactions` and `expenses` tables** and serialises them into the page.	`listTransactions()` = `select * from transactions` with no limit, passed as props to a client component. Every line carries ~40 fields. Payload and render time grow with every visit.	Aggregate in SQL (sales per month / branch / dentist / payment type) and send a few hundred numbers. Keep the detailed list on the paginated Transactions page.
2	**Missing indexes.** Only primary keys, a few foreign-key indexes and the status / patient indexes exist. Not indexed: `invoices.visit_date`, `invoices.branch_id`, `transactions.date`, `transactions.branch_id`, `transactions.visit_id`, `transactions.dentist_user_id`, `payments.paid_at`, `expenses.date`.	Day end filters `invoices.visit_date` and `payments.paid_at`; the list sorts by `transactions.date`.	Run `2026-10-performance-indexes.sql` (included with this review, not applied) and declare them in `schema.ts`.
3	**Every page navigation runs a count query in the layout** (`countOpenVisits()` for the Queue badge), and the menu then polls `/api/cashflow/open-visits` on a timer as well. The Queue page also calls `router.refresh()` every 30 s, which re-runs the whole page's queries.	`cashflow/layout.tsx`, `cashflow-nav.tsx`, `auto-refresh.tsx`	Drop the layout query (let the polled badge fill itself) or cache the count for 10–15 s; make the Queue refresh cheaper by polling a tiny "last changed" endpoint and refreshing only when it changed. With several front-desk screens open this removes most idle database traffic.
4	**Database round-trips.** Neon's HTTP driver costs one network round-trip per query (no transactions). Pages such as Transactions and Day end make 8–15 queries.	`transactions/page.tsx`, `db/day-end.ts`	Keep queries parallel (already mostly done); put the app and the Neon database **in the same region** (Singapore for a Philippine clinic); combine related lookups with joins; consider Neon's pooled WebSocket driver when transactions are needed. Note Neon's *auto-suspend*: the first request after idle can take 1–3 s (turn it off on a paid plan).
5	**Uploads stored as base64 text in Postgres.**	`attachments.data_base64`; download decodes the whole file in memory.	Move to object storage (Vercel Blob, Cloudflare R2 or S3) and keep only the URL / key in the table; serve through short-lived signed URLs. Smaller backups, faster queries, no memory spike.
6	**No caching of reference data.** Branches, dentists, procedures and price lists, discount types are re-read on almost every page although they change rarely.	no `unstable_cache` / `use cache` anywhere	Cache them for a few minutes with tags and call `revalidateTag` after the admin edits them. **Do this after the Next.js upgrade** (several advisories are about `use cache`).

3.2 Rendering and bundle

- **Very large client components:** `transaction-form.tsx` is 1,809 lines and `procedures-manager`, `dashboard-cards`, `charts`, `transactions-table` are 500–700 each; 45 files are `"use client"`. The Add-transaction form is mounted on the Queue, Transactions and Patient pages even when nobody opens it. Load modals and rarely used managers with `next/dynamic` (or render them only when opened) so each page ships less JavaScript; split the form into sections (patient, procedures, payment) with their own state to avoid re-rendering the whole form on every keystroke.

- **Charts** are hand-built (no chart library): good for size. Keep it that way.
- **Logo:** `logo-wordmark.png` is 1108 px wide (47 KB) but shown about 100–150 px wide. Export a 2× version (about 320 px, ~12 KB) or an SVG; use `next/image` or a plain smaller file. Remove the unused starter files in `public/` (`next.svg`, `vercel.svg`, `globe.svg`, `file.svg`, `window.svg`).
- **Loading and error states:** there are no `loading.tsx` files (pages block until all queries finish) and no `error.tsx`, `not-found.tsx` or `global-error.tsx` (a failure shows Next's raw "This page couldn't load"). Add skeleton `loading.tsx` for Transactions, Queue, Patients, Overview and Day end; add friendly error and 404 pages with a "try again" button and the clinic's contact.
- **Suspense streaming:** show the page frame and stat tiles first and stream the long tables (wrap them in `<Suspense>`).
- **Fonts:** the app uses the system font stack (no font downloads): already the fastest option.
- **Print:** invoice and day-end print styles exist (`print:hidden`); test A4 and thermal receipt widths.
- **Mobile:** the front desk may use tablets; check the 1,100 px-wide Day end table scrolls cleanly and the Queue works one-handed.

3.3 Measure before and after

1. **Lighthouse** (Chrome) on login, Overview, Transactions, Queue: record LCP, INP, CLS and total JS.
2. `@next/bundle-analyzer` to see which page ships the most JavaScript.
3. **Neon → Monitoring / Query insights**: the slowest queries and call counts (the polling will stand out).
4. **Vercel Speed Insights** (or Sentry Performance) in production for real-user numbers.
5. Load the database with realistic volume (for example 50,000 transaction lines: a seed script) and time Overview, Transactions and Day end *before* and *after* items 1–3 above.

3.4 Already good

Server-side pagination on Transactions and Patients; patient search done in the database with a prefix index; polling stops when the tab is hidden; parallel queries (`Promise.all`) on most pages; React Compiler rules enforced by lint; a navigation progress bar; no heavy chart / UI libraries; image uploads are shrunk in the browser before sending.

---

4. Operations: what a production system also needs

Area	Today	Suggestion
Error monitoring	none	Sentry (free tier) for browser and server errors, with patient data scrubbed.
Uptime and alerts	none	UptimeRobot / Better Stack on `/login`; alert by e-mail or Viber / SMS.
Staging site	none	A second deployment using the development database; try every release there first.
Automated checks	lint and type-check only; **no tests**	A CI step (GitHub Actions) running `tsc`, `eslint`, `yarn audit`, `build`. Add unit tests for the pure rules already written as separate modules (pricing, commission, visit status, transaction numbers, day end, session policy, patient HMO): they were checked by hand and by throw-away scripts, so turning those into permanent tests is cheap and prevents money-related regressions. A few Playwright tests for login → add visit → payment → Day end.

Area	Today	Suggestion
Releases	commits only	A short `CHANGELOG` and a version number shown in the footer, so a staff report can say which version.
Runbook	scattered in the docs	One page: how to deploy, roll back, restore a backup, rotate secrets, add a user, reset a password, run the SQL files in order.
Data fixes	admin must run SQL by hand in Neon	For repeated tasks (merge duplicates, delete a test visit), add admin screens with confirmation + audit; avoid direct SQL on live data.
Time zone	Manila logic exists in several places	Keep one helper; add a test around midnight and month ends.

---

5. Privacy, legal and "other things we missed"

> This is a checklist from the engineering side, not legal advice. The clinic should confirm the points marked (*) with a lawyer or the National Privacy Commission (NPC).

The system stores **sensitive personal information** under the Philippine **Data Privacy Act of 2012 (RA 10173)**: health-related information (procedures, HMO visits), government-issued ID numbers (PWD / Senior), birthdays and contact numbers. The clinic is the *personal information controller*; the developer / hosting provider, is a *processor* and should have a written data-processing agreement with the clinic (*).

5.1 Pages and notices to add

Item	Where	Notes
Privacy Policy	footer link on login and in the app	What is collected (patients and staff), why, who sees it, where it is stored (Neon / hosting region), how long it is kept, patients' rights (access, correction, erasure / blocking, complaint to NPC), contact of the **Data Protection Officer**.
Patient privacy notice and consent	printed or on-screen at registration / check-in; a "consent recorded" tick on the patient	The notice patients read when giving their details (including birthday and HMO number). Store *that* and *when* consent was given.
Terms of Use / acceptable use for staff	first sign-in (accept once, version recorded)	No sharing of accounts, no photos of screens, report lost devices.
Cookie notice	one line on the login page	Only an essential session cookie is used (no analytics or ads), so no consent banner is needed; say so.
FAQ / Help centre (`/cashflow/help`)	link in the menu and on login	See 5.2.
Contact / support page	footer	Who to call, hours, how to report a problem or a suspected breach.
404, error and "signed out" pages	app	See 3.2.

Item	Where	Notes
Accessibility statement (optional)	footer	The app already moves focus on route changes; document the target (WCAG 2.1 AA).

5.2 FAQ — suggested contents

Signing in: why am I signed out after 30 minutes; what is "Stay signed in"; I forgot my password; how do I get an account. *Daily work:* how do I add a visit to the Queue; what do Arrived / On-going / For Payment / Cleared mean; why can't I clear a visit (balance, procedure, dentist, **LOA pending**); how do I remove a visit from the Queue; what does the transaction number mean (`MMDDYYHHMM-BRANCH-CODE`); how do I record a payment, a reservation fee, a partial payment; PWD / Senior discount and VAT; card fees. *HMO:* what is an LOA / LOE and when can it be entered; why is an HMO visit "not paid"; where do I see what each HMO owes (Day end → HMO summary); where do I enter the patient's HMO account number. *Reports:* what is on Day end; how to download Excel / CSV; how commission is worked out (tiers, individual rate). *Mistakes:* I entered the wrong amount / patient; who can edit or delete; where is the history (audit trail). *Privacy:* who can see patient information; how do I report a mistake or a data request.

I can draft the Help page, Privacy Policy and Terms as ready-to-use text for the clinic to review; say the word.

5.3 Data protection tasks for the clinic (*)

1. **Appoint a Data Protection Officer** (a named person is required even for a small clinic).
2. **NPC registration:** the NPC requires registration of data-processing systems for organisations that process sensitive personal information of **1,000 or more individuals**, or employ 250 or more people (see NPC Circular 2022-04). With 120+ patients already on file after weeks of use, the clinic will likely pass 1,000 patients: check and register if so.
3. **Breach response plan:** a personal-data breach involving sensitive data must be reported to the NPC and the affected people **within 72 hours** of knowledge. Write a one-page incident procedure (who decides, who calls the vendor, what is recorded).
4. **Retention policy:** how long patient and financial records are kept (medical / tax rules apply), and what happens after (archive, anonymise). The system has no purge today.
5. **Data subject requests:** a procedure for "show me my data", "correct it", "delete it" (the patient profile and audit trail make access and correction easy; erasure of billing records may conflict with tax record-keeping).
6. **Staff confidentiality:** a signed confidentiality / data-protection undertaking per user; remove access the day someone leaves (needs the "disable user" feature).
7. **Vendor agreements:** written agreements with Neon and the hosting provider about data location and security.
8. **Data minimisation:** collect the birthday and HMO account only where needed; do not photograph IDs unless required.

5.4 Other product basics not built yet

- Patient **recall and reminders** by SMS / e-mail (recall dates are stored; sending is not built).
- **Receipts / official receipt numbering** rules and BIR requirements (*): check with the clinic's accountant.

- **Data export** for the accountant (monthly sales, VAT, expenses) in one click; **monthly closing** (lock a period so old figures can't be edited).
- **Role refinement:** a *Front desk* vs *Dentist* vs *Owner* distinction; per-branch managers.
- **Online / offline:** if the clinic's internet drops, the system cannot be used; define the paper fallback and how to enter it afterwards.
- **Training material:** a 10-minute video per role and a one-page quick-start card at each desk.
- **Open items from earlier:** rotate the Neon password and `AUTH_SECRET`; delete the test visit; run the pending SQL files.

---

6. Suggested roadmap

This week (small, high value)

1. Upgrade Next.js / Drizzle / dependencies; add Dependabot. Test on staging or the development database first.
2. Rotate the Neon password and `AUTH_SECRET`; create a Neon development branch and switch `.env.local` to it.
3. Run `2026-10-performance-indexes.sql`; declare the indexes in `schema.ts`.
4. Security headers + `robots.txt` / `noindex`; `error.tsx`, `not-found.tsx`.
5. Change the forgot-password page text to the honest version; add the admin **Reset password** button.

Next two weeks 6. Change-password and forced change at first sign-in; password blocklist; login throttling; sign-in audit entries; "disable user". 7. Overview computed in SQL (and scoped to branch); drop the layout count query and make the Queue refresh cheaper. 8. Privacy Policy, Terms, patient privacy notice, FAQ / Help page, support contact (draft together with the clinic's DPO). 9. Sentry, uptime monitor, a written backup + restore test.

This month and after 10. Move attachments to object storage; attachment branch check; file-type check. 11. Reference-data caching; lazy-load heavy modals; `loading.tsx` skeletons; image sizes. 12. CI with tests for the money rules; a few end-to-end tests; staging site; release notes. 13. Soft-delete and monthly closing; optional two-step verification for admins; versioned migrations instead of `db:push`.